



# Smart HAS

Monitoramento de Hipertensão Arterial Sistêmica

*App mobile (React Native) + API (Java/Spring Boot) + Painel web (Angular)*

**Documentação técnica da entrega**

**Integrantes do grupo**

Gabriel Garuti Paiva Cracco — RM 554866

## 1. Visão geral do projeto

O Smart HAS é uma solução para monitoramento da Hipertensão Arterial Sistêmica (HAS). O usuário registra medições de pressão (sistólica e diastólica), e o sistema classifica automaticamente cada medição como Normal, Elevada ou Hipertensão, além de gerar recomendações de acompanhamento e indicar a unidade de saúde mais próxima.

Nesta fase, a solução evoluiu para uma arquitetura unificada: uma única API REST central (Java + Spring Boot) passou a ser consumida por dois clientes distintos — um aplicativo mobile em React Native e um painel administrativo web em Angular. Assim, regras de negócio (como a classificação da pressão) ficam centralizadas no servidor e são compartilhadas por todos os clientes.

### Arquitetura

- Mobile (React Native / Expo) — usado pelo paciente para registrar e acompanhar medições.
- Painel web (Angular) — usado para administração: dashboard, medições e unidades de saúde.
- API central (Java / Spring Boot) — autenticação JWT, regras de negócio, persistência e documentação Swagger.
- Banco de dados relacional (H2), com troca simples para PostgreSQL/MySQL em produção.

Fluxo: os clientes autenticam-se em /auth/login, recebem um token JWT e o enviam no cabeçalho Authorization das próximas requisições. A API valida o token, aplica as regras de autorização por perfil (USER e ADMIN) e responde em JSON.

## 2. Parte 1 — Mobile: migração para React Native

### Decisão e justificativa

A fase anterior do Smart HAS possuía um aplicativo em Flutter (compilado para web) com backend em Python/FastAPI. Entre as opções da atividade — migrar para React Native ou aprimorar a base Flutter/Kotlin — optou-se pela migração para React Native.

Justificativas:

- Exercita diretamente os conceitos-alvo desta fase: componentes React Native (View, Text, Image, Button) e navegação com React Navigation.
- Permite que o mobile consuma exatamente a mesma API REST em Java/Spring Boot usada pelo painel Angular, resultando em uma arquitetura unificada (uma API, dois clientes).
- O contrato de dados do app original foi mantido (mesmos campos e rotas), tornando a migração fiel e de baixo risco.
- Como a Parte 2 já exige um backend em Java, reescrever a API e migrar o cliente mobile mantém todo o ecossistema coerente.

### O que foi implementado

- Autenticação (login) com persistência do token via AsyncStorage e Context API.
- Tela inicial com a última medição, sua classificação (com cor) e o nível de risco.
- Histórico de medições em lista (FlatList).
- Formulário de nova medição (TextInput + Button) integrado à API.
- Listagem de unidades de saúde.
- Navegação por abas (Bottom Tabs) e por pilha (Native Stack) com React Navigation.

## Componentes React Native utilizados

View (estrutura/layout), Text (textos), Image (logotipo do app), Button (ações de login e salvar), além de TextInput, FlatList, ScrollView e ActivityIndicator. A navegação usa @react-navigation/native com native-stack e bottom-tabs.

## Roadmap tecnológico

*Fase anterior: app Flutter (web) + backend Python/FastAPI + SQLite; recursos de mapa (Google Maps) e notificações já explorados.*

*Esta fase: mobile migrado para React Native (Expo); backend reescrito em Java/Spring Boot; novo painel administrativo em Angular — todos sobre a mesma API REST.*

*Próximas fases (planejado): publicação da API em nuvem, banco gerenciado (ex.: PostgreSQL ou Firebase), notificações push reais, build nativo do app (EAS Build) e evolução da camada de apoio à decisão (AI Logistics Extension) com histórico e tendências.*

### 3. Parte 2 — Back-end em Java + Spring Boot

A API foi construída com Spring Boot 3 (Java 17), seguindo a divisão em camadas: Controllers (web), Services (regras de negócio), Repositories (acesso a dados via Spring Data JPA) e Domain (entidades). Também há uma página administrativa renderizada no servidor com Spring MVC + Thymeleaf.

#### Entidades (modelo de dados)

| Entidade    | Campos principais                                                             |
|-------------|-------------------------------------------------------------------------------|
| User        | id, fullName, email (único), passwordHash, role (USER/ADMIN), createdAt       |
| Measurement | id, user (FK), systolic, diastolic, date, time, notes, createdAt              |
| HealthUnit  | id, name, type (HOSPITAL/CLINIC/SENSOR), latitude, longitude, address, active |

#### Endpoints da API REST

| Método | Rota               | Perfil  | Descrição                               |
|--------|--------------------|---------|-----------------------------------------|
| POST   | /auth/register     | Público | Cadastra um novo usuário                |
| POST   | /auth/login        | Público | Autentica e retorna um token JWT        |
| GET    | /measurements      | USER    | Lista as medições do usuário            |
| POST   | /measurements      | USER    | Cria uma medição                        |
| GET    | /measurements/{id} | USER    | Detalha uma medição                     |
| PUT    | /measurements/{id} | USER    | Atualiza uma medição                    |
| DELETE | /measurements/{id} | USER    | Remove uma medição                      |
| GET    | /units             | Público | Lista unidades de saúde                 |
| POST   | /units             | ADMIN   | Cria unidade                            |
| PUT    | /units/{id}        | ADMIN   | Atualiza unidade                        |
| DELETE | /units/{id}        | ADMIN   | Remove unidade                          |
| GET    | /recommendations   | USER    | Apoio à decisão (risco + recomendações) |
| GET    | /health            | Público | Verificação de saúde da API             |

#### Autenticação e autorização

A autenticação usa JWT (JSON Web Token). No login, a senha é conferida contra o hash BCrypt; em caso de sucesso, um token assinado (HS256) é retornado. Um filtro (JwtAuthFilter) intercepta cada requisição, valida o token e popula o contexto de segurança. A autorização é baseada em perfis: rotas de escrita de unidades exigem ADMIN (via @PreAuthorize e regras no SecurityConfig), enquanto medições ficam restritas ao próprio dono.

#### Banco de dados

Foi usado o H2 (banco relacional em arquivo), que roda sem instalação e facilita a correção/demonstração. O mapeamento objeto-relacional é feito com JPA/Hibernate (spring-boot-starter-data-jpa). Para produção, basta trocar as propriedades de datasource por PostgreSQL ou MySQL. O console web do H2 fica disponível em /h2-console.

#### CRUD, validações e tratamento de erros

As medições possuem CRUD completo (criar, listar, detalhar, atualizar e remover). A validação de entrada usa Bean Validation (@Valid, @NotBlank, @Min/@Max etc.). Erros são tratados de forma central por um @RestControllerAdvice (GlobalExceptionHandler), que devolve um JSON padronizado com o campo "detail", mantendo compatibilidade com os clientes.

## **Documentação (Swagger/OpenAPI)**

A API é documentada automaticamente com springdoc-openapi. A interface interativa fica em [/swagger-ui.html](#), incluindo o esquema de autenticação Bearer (JWT) para testar as rotas protegidas.

## **Camada de apoio à decisão (AI Logistics Extension)**

O endpoint `/recommendations` analisa o histórico do paciente, calcula um nível de risco (BAIXO, MODERADO ou ALTO) a partir das medições recentes, gera recomendações textuais e sugere a unidade de saúde ativa mais próxima (cálculo de distância pela fórmula de Haversine).

## 4. Parte 3 — Painel web em Angular

O painel administrativo foi desenvolvido em Angular (componentes standalone) e consome a mesma API REST via HttpClient. Um interceptor adiciona automaticamente o token JWT no cabeçalho das requisições.

### Telas e rotas

- /login — autenticação (formulário com [(ngModel)]).
- /dashboard — resumo do paciente, nível de risco e recomendações.
- /measurements — histórico e cadastro de medições (CRUD).
- /units — listagem e gestão de unidades de saúde (ADMIN).

### Conceitos de Angular aplicados

- Data binding: interpolação {{ }}, property binding [ ], event binding ( ) e two-way binding [( )] com ngModel.
- Diretivas estruturais: \*ngIf (exibição condicional) e \*ngFor (listas).
- Formulários com FormsModule e [(ngModel)] nas telas de login, medições e unidades.
- Roteamento com provideRouter e navegação por routerLink.
- Serviços injetáveis (ApiService e AuthService) e HttpClient para consumir a API.
- Estilização com CSS global e classes reutilizáveis (cards, badges, tabelas).

## 5. Como executar

### 1) Back-end (porta 8080)

- Requisitos: Java 17+ e Maven (ou IDE como IntelliJ).
- Comando: `cd backend-springboot && mvn spring-boot:run`
- Swagger: `http://localhost:8080/swagger-ui.html` | Visão geral (Thymeleaf): `http://localhost:8080/`
- Usuários demo: `admin@smarthas.com` / `admin123` (ADMIN) e `paciente@smarthas.com` / `123456` (USER).

### 2) Painei Angular (porta 4200)

- Requisitos: Node.js 18+ e Angular CLI.
- Comandos: `cd web-angular && npm install && npm start`

### 3) App React Native (Expo)

- Requisitos: Node.js 18+ e Expo Go (ou emulador).
- Comandos: `cd mobile-react-native && npm install && npx expo start`
- Ajuste `API_URL` em `src/api/client.js` conforme o ambiente (emulador Android: 10.0.2.2; celular físico: IP da máquina).

## 6. Links e entrega

- Repositório GitHub: (inserir link)
- Vídeo de demonstração (YouTube, não listado): (inserir link)

*Observação: os nomes, RMs e o vídeo devem ser preenchidos pelo grupo antes da entrega final.*